

DOSSIER PROFESSIONNEL (DP)

Nom de naissance ▶ LESIEUR
Nom d'usage ▶
Prénom ▶ DYLAN
Adresse ▶ CALLE GALERA 39, MADRID, 28042

Titre professionnel visé

DEVELOPPEUR WEB ET WEB MOBILE

MODALITE D'ACCES :

- ☒ Parcours de formation
- ☐ Validation des Acquis de l'Expérience (VAE)

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel. **Ce titre est délivré par le Ministère chargé de l'emploi.**

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen.**

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.

Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]

Ce dossier comporte :

- ▶ pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- ▶ un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- ▶ une déclaration sur l'honneur à compléter et à signer ;
- ▶ des documents illustrant la pratique professionnelle du candidat (facultatif)
- ▶ des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.



<http://travail-emploi.gouv.fr/titres-professionnels>

Sommaire

Exemples de pratique professionnelle

Développer la partie front-end d'une application web ou web mobile sécurisée p. 5

- ▶ Installer et configurer son environnement de travail en fonction du projet web ou web mobile p. 5
- ▶ Maquetter des interfaces utilisateur web ou web mobile p. 8
- ▶ Réaliser des interfaces utilisateur statiques web ou web mobile p. 11

Développer la partie back-end d'une application web ou web mobile sécurisée p. 15

- ▶ Mettre en place une base de données relationnelle p. 16
- ▶ Développer des composants d'accès aux données SQL et NoSQL p. 20
- ▶ Développer des composants métier côté serveur p. 25

Titres, diplômes, CQP, attestations de formation *(facultatif)* p. 30

Déclaration sur l'honneur p. 31

Documents illustrant la pratique professionnelle *(facultatif)* p. 32

Annexes *(Si le RC le prévoit)* p. 33

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Activité-type 1

Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°1 ► *Installer et configurer son environnement de travail en fonction du projet web ou web mobile*

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Lorsque j'ai démarré le projet Vite Gourmand, la toute première étape a été de mettre en place un environnement de développement. C'était une des conditions préalables pour travailler efficacement et garantir que l'application puisse être relancée de la même manière sur n'importe quelle machine, évitant ainsi l'effet « ça marche chez moi mais pas chez les autres ». Le but du projet était de créer une application web full-stack pour un traiteur gastronomique.

Analyse de l'environnement technique et lecture des spécifications

Avant d'installer quoi que ce soit, j'ai cartographié (voir Fig. 15) l'ensemble des composants techniques :

- front-end (React, Vite, TypeScript)
- back-end (NestJS, Prisma)
- deux bases de données (PostgreSQL [Supabase], MongoDB [Atlas])
- des services tiers (SMTP, API Unsplash, IA).

Cette lecture depuis la mindmap m'a permis de prévoir les variables d'environnement nécessaires avant d'écrire la première ligne de code.

Installation des outils de développement

Sous Linux Ubuntu (machine virtuelle), j'ai installé Visual Studio Code avec des extensions comme : TypeScript, ESLint, Prettier, Tailwind CSS IntelliSense, Node.js LTS 22, les clients PostgreSQL et les outils Prisma en ligne de commande.

Conteneurisation avec Docker

J'ai rédigé un fichier docker-compose.yml définissant un service unique basé sur Node.js 22 Alpine, exécutant les deux serveurs dans le même conteneur, avec des volumes nommés pour persister les node_modules ainsi que des healthchecks au démarrage. Un Makefile orchestre toutes les opérations courantes avec les cibles suivantes : all, bootstrap, local, deploy, test, secrets, stop, shell, fclean, help (voir Fig. 8).

42madrid@fundaciontelefonica.com

Gestion des secrets avec Bitwarden CLI

Un script s'appuyant sur Bitwarden CLI récupère automatiquement les secrets (chaînes de connexion, clés JWT, identifiants SMTP, tokens d'API) depuis le coffre-fort au lancement des conteneurs et les injecte

comme variables d'environnement. Le fichier **.gitignore** est configuré pour exclure les fichiers générés et les potentielles fuites d'informations sensibles.

Versionnage et intégration continue

J'ai initialisé le dépôt Git, configuré un dépôt sur GitHub et mis en place des workflows GitHub Actions pour vérifier automatiquement à chaque push la compilation TypeScript, les tests unitaires, ainsi que SonarCloud (une plateforme QA) (voir Fig. 11) qui analyse les failles de sécurité, la maintenabilité et les bonnes pratiques. Tout cela est visible depuis la Fig. 14. De nouveaux jobs ont été ajoutés à chaque itération pour vérifier automatiquement la bonne santé des branches.

Difficultés rencontrées

La connexion à Supabase depuis le conteneur nécessitait un paramétrage spécifique : les options `pgbouncer=true` et `connection_limit=1`, identifiées par l'analyse des journaux d'erreurs (voir Fig. 20).

Vérification de l'environnement

Une fois l'environnement enfin opérationnel, j'ai vérifié le démarrage du front-end, réponse de l'API NestJS avec documentation Swagger accessible, la compilation TypeScript sans erreur, et la connexion effective aux deux bases de données via des endpoints réels.

Refactorisation de l'infrastructure

Au fil du développement, j'ai refactorisé l'organisation des services Docker dans une arborescence dédiée (voir Fig. 9 et Fig. 17)

, séparant clairement les responsabilités entre les services applicatifs, les bases de données et les outils ponctuels (Bitwarden CLI, Fly CLI). Cette séparation a éliminé les conflits de configuration et rendu chaque service indépendant et remplaçable. Bien que l'approche monolithique soit souvent recommandée pour un projet de cette taille, les incompatibilités entre les différents postes de travail ont motivé le choix de cette infrastructure.

Cette organisation est plus exigeante à maintenir, mais elle garantit une reproductibilité totale de l'environnement.

Quant au Makefile, il a été structuré en modules dans un répertoire `mk_extensions/`, chaque fichier regroupant les cibles par domaine. Cette modularité permet d'ajouter de nouvelles cibles sans alourdir le fichier principal, qui joue le rôle d'une documentation exécutable.

Corrections de la chaîne d'intégration continue

La configuration de GitHub Actions a nécessité plusieurs corrections pour être suffisamment opérationnelle. Un problème de configuration TypeScript générait des chemins `rootDir` incorrects et empêchait la compilation en CI. Cela a été résolu en ajustant les chemins dans `tsconfig.json`. En fait, Jest nécessitait l'option `--localstorage-file` pour isoler le stockage entre les tests en environnement CI, évitant ainsi les interférences entre suites de tests parallèles.

2. Précisez les moyens utilisés :

DOSSIER PROFESSIONNEL (DP)

Visual Studio Code : extensions TypeScript, ESLint, Prettier, Tailwind CSS IntelliSense.

Node.js 22 LTS : environnement d'exécution pour NestJS et Vite ; Docker et Docker Compose : conteneurisation sur image Node.js 22 Alpine.

Makefile : interface unifiée (démarrage, arrêt, logs, nettoyage) ; Bitwarden CLI : injection sécurisée des secrets au démarrage.

Git et GitHub : contrôle de version, dépôt distant, GitHub Actions pour la CI TypeScript.

Prisma CLI : gestion du schéma et génération du client ; Supabase (PostgreSQL) et MongoDB Atlas : bases de données cloud.

3. Avec qui avez-vous travaillé ?

Ce projet a été développé en autonomie complète. La résolution des problèmes techniques a reposé sur la documentation officielle (Docker, NestJS, Prisma, Vite), les forums spécialisés (Stack Overflow, Discord NestJS et Prisma) et les journaux d'erreur des outils. Aucun collaborateur externe n'est intervenu sur le code.

4. Contexte

Nom de l'entreprise, organisme ou association ► **Formation autonome — Projet Vite Gourmand**

Chantier, atelier, service ► Développement full-stack en autonomie

Période d'exercice ► Du 01/01/2026 au 15/02/2026

5. Informations complémentaires (facultatif)

La conteneurisation Docker garantit la reproductibilité de l'environnement sur toute machine. Le Makefile centralise toutes les opérations disponibles. La gestion des secrets via Bitwarden CLI assure qu'aucune donnée sensible n'est jamais versionnée.

Activité-type 1

Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°2 ▶ Maquetter des interfaces utilisateur web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans ce même projet, j'ai eu la responsabilité de m'occuper de la conception et de la mise en forme des interfaces de l'application, qui s'adressait à deux publics distincts : des clients et des professionnels.

Analyse des besoins utilisateurs

Avant de composer mes premiers éléments UI, j'ai analysé les besoins propres à chaque type d'utilisateur. Pour le grand public, l'interface devait évoquer un environnement gastronomique et premium avec une grande facilité de navigation.

Pour le public professionnel, elle devait prioritairement être pratique, dans le sens où les professionnels voudront manipuler les données et interagir rapidement avec les clients.

J'ai traduit ces exigences en choix d'identité visuelle concrets : une palette centrée sur le bordeaux profond, le champagne doré, le vert olive et le crème, et deux familles typographiques. Pour la police, j'ai choisi Inter pour la lisibilité à toutes les tailles, et Playfair Display pour les titres de la vitrine publique.

Croquis et structure des pages

J'ai ensuite travaillé à main levée pour poser la structure générale des pages. Ces croquis que j'ai faits représentaient plus tard la page d'accueil, l'organisation du catalogue de menus, et la navigation entre la vitrine et le portail d'administration. Ces croquis basse-fidélité m'ont permis de valider la logique de navigation avant d'investir du temps dans les détails visuels (voir Fig. 1 et Fig. 2).

Je connaissais Figma, mais après plusieurs jours à apprendre tous ses outils, je n'avançais toujours pas sur le projet. J'avais besoin de simplifier le processus. C'est pourquoi j'ai choisi de travailler avec Excalidraw. Simple et puissant c'est un outil polyvalent que l'on peut facilement utiliser pour créer des maquettes basse-fidélité.

Conception des parcours mobiles

J'ai accordé une attention particulière aux parcours mobiles : l'ordre des sections sur un écran vertical, la position des boutons d'action à portée de pouce, la transformation du menu de navigation en panneau plein écran.

Système de tokens CSS

La décision du projet a été de construire l'ensemble du design sur un système de tokens CSS avant de concevoir la moindre interface. J'ai organisé ce système en trois fichiers de charte graphique (voir Fig. 16) :

***le premier** concentrant tous les tokens de couleur (dix niveaux de teinte par couleur sémantique, couleurs de fond légèrement teintées de violet, couleurs de texte exprimées en opacités).*

DOSSIER PROFESSIONNEL (DP)

le deuxième regroupant les tokens de construction (échelle typographique en huit niveaux, quatre graisses de police, treize valeurs d'espacement, six niveaux de rayon de bordure, cinq intensités d'ombres portées).

le troisième gérant les tokens de comportement (huit niveaux de z-index, trois vitesses de transition, dimensions normalisées des composants, tokens de mise en page).

Conception de la page Mentions légales et cookies

La conformité CNIL exigeait une page dédiée à la transparence sur les cookies.

Un tableau lisible présentant les sept cookies utilisés (Nom, Usage, Durée, Type)

des badges colorés (Essentiel, Fonctionnel, Tiers),

un bloc "Comment changer d'avis" expliquant comment révoquer le consentement,

un bloc "Vos droits RGPD" listant les droits (accès, rectification, effacement, opposition, portabilité) avec un lien direct vers le formulaire CNIL.

L'objectif : qu'un utilisateur non-technique comprenne en moins d'une minute ce qui est stocké, pourquoi, et comment l'effacer.

2. Précisez les moyens utilisés :

Croquis papier : structuration basse-fidélité des parcours et mises en page avant toute implémentation.

Excalidraw : structuration basse-fidélité plus dynamique et mise en place des relations.

Variables CSS natives : système en trois fichiers de charte graphique couvrant couleurs, typographie, espacement, ombres, transitions et dimensions de composants.

Tailwind CSS 4 + CVA : framework utilitaire avec palette de marque personnalisée

Radix UI : primitives accessibles (ARIA, focus, clavier) stylisées avec les tokens du design system ; Lucide

React : icônes SVG importées unitairement ; @axe-core/react : audit d'accessibilité en développement.

DevTools Chrome : inspection des variables CSS, simulation mobile, vérification des contrastes ; documentation Radix UI, Tailwind CSS, CVA et MDN pour les fonctions CSS avancées.

3. Avec qui avez-vous travaillé ?

DOSSIER PROFESSIONNEL (DP)

Ce projet a été conduit en totale autonomie, m'amenant à occuper simultanément les rôles de designer et de développeur. Chaque choix de design était immédiatement évalué sous l'angle de sa faisabilité et de sa maintenabilité techniques. La conception s'est faite directement dans le navigateur, les DevTools jouant le rôle d'outil de validation visuelle. Des ressources communautaires (documentation Radix UI, forums Tailwind, Discord front-end) ont ponctuellement servi de sources de validation.

4. Contexte

Nom de l'entreprise, organisme ou association **Formation autonome — Projet Vite Gourmand**

Chantier, atelier, service ▶ Développement full-stack en autonomie

Période d'exercice ▶ Du 01/01/2026 au 15/02/2026

5. Informations complémentaires (facultatif)

Le système de tokens CSS a prouvé son efficacité lors de l'ajout du tableau de bord : tous les nouveaux composants ont été construits en quelques heures sans créer de nouveaux tokens. Une modification de valeur se propage instantanément sur l'ensemble de l'interface. Ce qui constitue une preuve concrète d'un design system maintenu par convention plutôt que par copie.

Activité-type 1

Exemple n°3 ▶

Développer la partie front-end d'une application web ou web mobile sécurisée

Réaliser des interfaces utilisateur statiques et développer la partie dynamique des interfaces utilisateur web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre de ce projet, j'ai réalisé l'intégralité des interfaces utilisateur statiques de l'application web, couvrant cinq pages publiques principales : la page d'accueil, le catalogue des menus, la page de contact, la page de commande et les pages légales.

Structure sémantique HTML

J'ai porté une attention particulière à la structure sémantique de chaque page :

- balise `<nav>` pour la barre de navigation;
- `<header>` pour l'en-tête;
- `<main>` pour le contenu principal;
- `<section>` pour chaque bloc thématique;
- `<article>` pour chaque carte de menu ou de témoignage;
- et `<footer>` pour le pied de page.

- ...

- Le fichier HTML racine déclare explicitement la langue française (`lang="fr"`), le jeu de caractères UTF-8, et la balise `viewport` indispensable au rendu mobile. Les titres respectent une hiérarchie stricte : un seul `<h1>` par page, des `<h2>` pour les titres de sections, des `<h3>` pour les titres de cartes.

Mise en forme avec Tailwind CSS

Pour la mise en forme, j'ai utilisé Tailwind CSS 4 en combinaison avec le système de variables CSS personnalisées défini lors de la phase de maquettage. La mise en page s'appuie sur les grilles CSS et Flexbox via les utilitaires Tailwind, avec une approche mobile-first systématique : chaque grille est d'abord conçue en colonne unique sur mobile, puis enrichie par paliers pour les écrans plus larges.

Optimisation des médias

Les images proviennent d'Unsplash, qui les optimise à la demande via des paramètres d'URL (format, dimensions, qualité). Les icônes Lucide React sont importées de manière unitaire pour que le bundler Vite n'inclue que celles effectivement utilisées.

Accessibilité

Dans un monde de plus en plus inclusif, l'accessibilité par design est un choix primordial qui améliore substantiellement la qualité d'une webpage. C'est pourquoi les composants interactifs complexes (modales, menus déroulants, accordéons) sont construits sur Radix UI, qui gère nativement les attributs ARIA, la gestion du focus et la navigation clavier. Quant à l'état de focus, il est visible et défini globalement pour tous les éléments interactifs. De plus la sémantique html, si respectée, est un fondamental pour l'accessibilité web. Par exemple, les lecteurs d'écran comme NVDA ou VoiceOver naviguent principalement via la structure sémantique. Un `<nav>` annonce "zone de navigation", `<main>` permet de sauter directement au contenu principal, etc.

Concrètement dans le projet: si on a un `<article>` pour chaque carte de menu, un lecteur d'écran peut lister tous les articles de la page et naviguer entre eux directement. Si l'on avait mis des `<div>` partout à la place, cette navigation serait impossible sémantique à moins que l'on utilise les attributs aria.

Tests et vérifications

Une fois chaque interface réalisée, j'ai procédé à une vérification systématique : test d'affichage sur les principales résolutions via les DevTools Chrome, vérification de la navigation au clavier sur chaque page, et audit Lighthouse pour évaluer les performances, l'accessibilité, le SEO, et les bonnes pratiques. Ces tests ont guidé la construction du site web.

Sécurité des interfaces utilisateur et conformité CNIL

La sécurité front-end a été traitée au travers de 11 problèmes répertoriés dans le code space.

La principale faille, était le JWT stocké dans localStorage, accessible depuis JavaScript et donc exposé aux attaques XSS. La stratégie pour réparer cette issue était de migrer vers une stratégie cookie HttpOnly + Secure rendant le token inaccessible au code client. Pour renforcer la sécurité un mécanisme CSRF via l'en-tête X-CSRF-Token a aussi été implémenté. Ainsi, le serveur émet le token dans un cookie lisible que le client relit et renvoie dans chaque requête mutante.

Qualité et référencement

Six routes principales ont été auditées avec Lighthouse (Performance, Accessibilité, Bonnes pratiques, SEO), obtenant des scores supérieurs à 90 sur chacune (voir Fig. 12 et Fig. 13). Pour le référencement naturel, j'ai mis en place un fichier robots.txt autorisant l'indexation de l'ensemble du site (Allow: /), un sitemap.xml listant les six URL indexables (voir Fig. 18). Selon <https://developers.google.com/docs/crawling-indexing/robots/intro>. C'est un fichier de texte que l'on va mettre à la racine de notre site pour communiquer aux robots des moteurs de recherche comme googlebot. Il faut imaginer ce fichier comme un panneau de direction que liraient les robots. Pour chaque index à faire, il lit les instructions à respecter. Dans notre cas, on a simplement bloqué des pages pour l'indexation et amélioré le référencement SEO. Sans ce fichier, Google indexe et interprète ce qu'il veut. À la place, on peut décider de lui donner la carte de notre site web à visiter. Pour éviter que les pages admins n'apparaissent dans le référencement naturel. On peut tout simplement Disallow.

Ce que j'ai surtout appris, c'est qu'il ne faut pas confondre ce robots.txt avec une sécurité. C'est juste une convention. Un robot mal intentionné peut tout simplement l'ignorer. Les routes admins restent accessibles à n'importe qui saisit l'URL entière. La vraie protection, c'est l'utilisation des tokens JWT dans NestJS.

Concrètement, quelqu'un qui cherche « traiteur paris », Google consultera sa base de données pour afficher un lien vers d'autre site web qui ne connaîtront probablement pas vite-gourmand car ils ne sont pas basés à Paris.

2. Précisez les moyens utilisés :

Visual Studio Code : extensions TypeScript, ESLint, Prettier, Tailwind CSS IntelliSense ; Vite 7 : serveur de développement HMR, bundler Rollup en production.

HTML5 sémantique : balises de structure (nav, header, main, section, article, footer), hiérarchie des titres, attributs lang et viewport.

Tailwind CSS 4 + variables CSS natives : framework utilitaire mobile-first avec tokens de couleurs, typographie et espacement en trois fichiers de charte graphique ; CSS clamp() pour la typographie fluide.

Radix UI (composants accessibles ARIA/focus/clavier), Lucide React (icônes SVG unitaires), @axe-core/react (audit accessibilité en développement).

Lighthouse + DevTools Chrome : performances, accessibilité, SEO ; Unsplash CDN : images optimisées à la demande ; Git et GitHub : versionnement.

3. Avec qui avez-vous travaillé ?

DOSSIER PROFESSIONNEL (DP)

Projet conduit en autonomie complète. Seul à occuper les rôles de designer, intégrateur et développeur front-end, j'ai validé mes décisions via les outils d'audit automatisé (Lighthouse, axe-core), les DevTools pour le comportement responsive, et la documentation officielle W3C. Les forums communautaires (Discord Tailwind, documentation Radix UI) ont servi de référence pour les cas non couverts par la documentation.

4. Contexte

Nom de l'entreprise, organisme ou association **Formation autonome — Projet Vite Gourmand**

Chantier, atelier, service ▶ Développement full-stack en autonomie

Période d'exercice ▶ Du 01/01/2026 au 15/02/2026

5. Informations complémentaires (facultatif)

Éco-conception : chargement différé des images et import unitaire des icônes réduisent les données téléchargées. Sécurité formulaires : validation côté client (HTML natif, required/maxlength) couplée à la validation côté serveur via class-validator pour une couverture complète.

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°1 ► Mettre en place une base de données relationnelle

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre de ce projet l'une des tâches les plus structurantes du back-end a été la conception et la mise en place de la base de données relationnelle. L'application couvre en réalité bon nombre de métiers variés — gestion des commandes, fidélité client, menus avec allergènes, messagerie interne, support, ressources humaines, et j'en passe. La base de données devait être pensée dès le départ comme un système cohérent, évolutif et sécurisé.

Recueil des besoins et analyse conceptuelle

Avant de définir la moindre table, j'ai listé l'ensemble des entités métier nécessaires. Ce travail d'analyse m'a permis d'identifier quarante-six modèles de données distincts, regroupés en grandes familles thématiques comme (utilisateurs et authentification, gestion de l'entreprise, menus et plats, commandes et livraisons, etc.). Le diagramme étant beaucoup trop grand pour un simple screenshot dans word (voir Fig. 5); j'ai donc dû omettre cette partie. Pour chaque entité, j'ai défini les attributs nécessaires, les types de données appropriés, les contraintes d'intégrité et les relations avec les autres entités.

Définition du schéma physique avec Prisma

Bon nombre d'ORM existent. J'ai finalement choisi Prisma qui dans le futur m'a inspiré pour créer un autre projet appelé Prismatic. C'est un ORM servant à la fois de modélisation, de générateur de client typé et de gestionnaires de migrations. C'est-à-dire, grâce à Prisma, on va décrire les tables en TypeScript dans un seul fichier appelé schema.prisma. Ici on écrira jamais de SQL. Prisma lit ce fichier et comprend la structure de la base. À partir de ce schéma, Prisma génère automatiquement du code TypeScript que l'on peut utiliser directement. Quand on modifie le schéma comme par exemple ajouter une colonne ou renommer une table. Prisma génère un fichier de migration. Chaque fichier de migration contient le SQL exact qui a modifié la base. On peut rejouer l'historique. Revenir en arrière, ou appliquer les mêmes changements sur un autre serveur. Sans cet ORM on aurait dû faire les trois à la main: écrire le SQL de création des tables, écrire les requêtes SQL dans le code, gérer les scripts de migration moi-même.

Conception des relations entre tables

L'application repose sur trois types de relations. Les relations un-à-plusieurs sont les plus fréquentes : un utilisateur peut passer plusieurs commandes, un compte de fidélité peut comporter de nombreuses transactions (voir Fig. 3).

Ma démarche a été délibérément top-down : je ne me suis pas mis à créer des tables dès le départ, mais j'ai d'abord cartographié l'ensemble des processus métier (commande, fidélité, gestion des menus, ressources humaines, messagerie, support). Ce recensement a fait émerger les entités et leurs liens

avant que le moindre schéma Prisma ne soit écrit. Cette phase d'analyse préalable m'a évité les restructurations tardives qui coûtent cher en migrations.

Pour les relations plusieurs-à-plusieurs — tags d'allergènes sur les plats, rôles sur les utilisateurs, promotions sur les commandes — j'ai systématiquement créé des tables de jonction explicites plutôt que de m'appuyer sur les relations implicites de Prisma. Ce choix laisse la possibilité d'enrichir la relation d'attributs métier supplémentaires (date d'application d'une promotion, source d'attribution d'un rôle) sans restructurer le schéma. Les relations un-à-un, plus rares, ont été utilisées pour isoler les données sensibles : le profil de fidélité est séparé du compte utilisateur de façon à pouvoir être supprimé indépendamment en cas de demande RGPD.

Sur la question de la normalisation, j'ai appliqué une règle claire : tout ce qui doit rester cohérent et interrogeable avec des contraintes transactionnelles va dans PostgreSQL sous forme normalisée ; tout ce qui est une photographie figée d'un état passé (instantané de commande, statistiques agrégées, journal d'audit) va dans MongoDB sous forme dénormalisée. Cette séparation évite les jointures lourdes sur des données historiques tout en garantissant l'intégrité des données actives.

Enfin, l'utilisation de Prisma Migrate a permis de faire évoluer le schéma de façon itérative et traçable. Chaque modification — ajout d'un champ, renommage d'une relation, introduction d'un index partiel — a généré un fichier de migration versionné. Cela m'a permis de rejouer fidèlement l'historique du schéma sur un nouvel environnement et de revenir en arrière en cas d'erreur, sans jamais toucher directement à la base.

Mise en place des index

Trente-quatre index ont été définis sur les colonnes fréquemment sollicitées : colonnes de statut des commandes, des avis et des promotions pour les filtres ; colonnes de date pour les tris chronologiques ; clés étrangères les plus utilisées. Certains index sont partiels. L'index sur les abonnés actifs à la newsletter ne porte que sur les enregistrements dont `is_active` vaut vrai. Ce qui réduit la taille de l'index et améliore les performances des requêtes courantes.

Les index peuvent parfois être confondus avec d'autres mécanismes : il faut bien distinguer un index SQL (structure interne à la base qui accélère les requêtes côté serveur) d'une indexation par un moteur de recherche comme Google (référencement public d'une page sur le web). Les deux n'ont aucun rapport et n'agissent pas sur les mêmes performances.

Sécurité et confidentialité des données

La sécurité des données a été traitée à plusieurs niveaux. Premièrement, les mots de passe sont hachés avec `bcrypt` à douze tours de salage. La politique du mot de passe a été renforcée (longueur minimale de huit caractères, majuscule, minuscule, chiffre), appliquée lors du changement de mot de passe. De plus, les tokens JWT ont une durée de vie de quinze minutes. Les tokens de réinitialisation de mot de passe sont des chaînes aléatoires de trente-deux octets, stockés avec une expiration d'une heure et marqués comme utilisés après leur première consommation.

Au départ, cela m'avait beaucoup coûté de comprendre comment fonctionne le chiffrement. En réalité, pendant des années et encore aujourd'hui, le chiffrement est sûrement l'un des plus grands défis que les développeurs n'aient jamais connu, un peu comme la boîte de Pandore. Voici comment je l'ai compris, du plus risqué au moins risqué.

Le problème de départ c'est que si les développeurs stockaient les mots de passe en clair, la personne

qui accède à la base obtient directement tous les mots de passe. Pendant longtemps, le hachage simple a été utilisé dans la communauté : MD5 et SHA-256.

Si on utilise SHA256("test123 !") qui aurait une sortie "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx" en hexadécimal. Un attaquant peut simplement créer une table de hachage, les précalculer à l'avance et récupérer les mots de passe en quelques millisecondes. On appelle cela une table arc-en-ciel (rainbow table). C'est encore plus évident si plusieurs personnes ont le même hash. Le modèle est alors trop prédictible. Le salage quant à lui est une valeur aléatoire unique générée pour chaque utilisateur, ajoutée au mot de passe avant de hacher.

Cela génère des hachages totalement différents pour le mot de passe. Le rainbow table devient inutile et l'attaquant aurait dû générer des milliards et des milliards de valeurs puisqu'il devrait calculer pour chaque sel. Cependant, le sel n'est pas un secret et il est facile de retrouver le mot de passe puisque le sel est contenu dans le hash. Même avec un sel, cela reste trop rapide de trouver le mot de passe puisqu'un GPU peut tester des milliards et des milliards de combinaisons par seconde.

Bcrypt résout ça avec un facteur de coût qui rend le calcul intentionnellement lent (voir Fig. 19). Dans le projet on a utilisé $2^{12} = 4096$ itérations de hachage par mot de passe. ~300ms pour le projet. Selon OWASP, il est conseillé de mettre au moins 10 ou sinon 12 pour plus de renfort. Si l'on choisit 14, le temps serait 4 fois plus lent par mot de passe que le 12. Soit environ 1200 ms. Pour un utilisateur qui se connecte c'est 300ms. Pour un attaquant c'est 9,5 ans de calcul.

$1\ 000\ 000\ 000 * 0.3\ \text{sec}$

$= 3\ 000\ 000\ \text{secondes}$

$= 9,5\ \text{ans (par cœur CPU)}$

Le sel est stocké en base (visible si la BDD est volée). Le poivre est un secret côté serveur, jamais en base.

`Hash = bcrypt(mot de passe + PEPPERSECRET, SALTROUND)`

Dans le projet, le poivrage n'est pas implémenté. password.service.ts appelle directement `bcrypt.hash(password, SALT_ROUNDS)` sans ajouter de secrets serveur. C'est assumé et ce n'est pas critique puisque bcrypt + sel est déjà très solide. Mais c'est une vraie couche de défense en profondeur qui manque.

Cela veut dire que si l'attaquant vole uniquement la base de données, il ne peut pas cracker les hashes sans connaître le poivre stocké dans les variables d'environnement du serveur.

Note : une solution brute force serait d'augmenter la capacité de calcul des composants matériels. Si un attaquant venait à attaquer il aurait besoin de GPU. Une RTX 4090 sur bcrypt ferait environ 12 000 hachages/seconde.

$1\ 000\ 000\ 000 / 12\ 000 = 83\ 333\ \text{secondes} \approx 23\ \text{heures}$.

Avec 10 GPU, ce serait environ 2 heures.

Avec une ferme de GPU, probablement quelques minutes.

Ce n'est pas viable. Aujourd'hui ils utilisent des dictionnaires. Un mot de passe comme 123456 mettrait quelques ms à être découvert. Un mot de passe avec Test123!, probablement quelques secondes. Un

mot de passe plus complexe comme celui-ci "@#~#@½|@~#~½#~{~½{" prendrait beaucoup plus de temps.

À cela s'ajoute une politique de mot de passe + rate limiting NestJS avec par exemple 20 requêtes/sec max. + mot de passe récupéré par e-mail.

Conformité RGPD

Quand un utilisateur clique sur "j'accepte", on ne se contente pas d'afficher un bandeau et d'oublier. Chaque consentement est enregistré en base avec trois informations. Ce qui a été accepté, quand, et depuis qu'elle adresse IP. Si un juge ou la CNIL demande un jour si un utilisateur a bien consenti. La réponse est traçable. Quand quelqu'un demande la suppression de son compte, on ne supprime pas la ligne en base. On fait ce qu'on appelle l'anonymiser. Son courriel devient une valeur générique, les attributs liés à son identité sont effacés. Ses commandes, elles sont conservées : une entreprise a l'obligation légale de garder ses données comptables plusieurs années, même si le client part. La demande de suppression elle-même est archivée, avec la date à laquelle elle a été traitée et l'identifiant de l'admin qui l'a exécutée.

Mise en place des données de test

Pour permettre le développement et les tests dans des conditions réalistes, j'ai créé un jeu de données complet réparti en quarante-cinq fichiers SQL : vingt utilisateurs représentant tous les rôles, trente-cinq menus, vingt commandes avec statuts variés, comptes de fidélité, avis et tickets de support. Les mots de passe des utilisateurs de test sont hachés avec bcrypt, exactement comme en production. En complément, un pipe de validation global (class-validator, class-transformer) inspecte chaque requête entrante, rejette les propriétés non déclarées et retourne une réponse listant chaque champ invalide avec le motif de rejet.

2. Précisez les moyens utilisés :

DOSSIER PROFESSIONNEL (DP)

PostgreSQL (Supabase cloud) + Prisma ORM : modélisation du schéma, génération du client TypeScript typé, gestion des migrations via Prisma CLI ; adaptateur @prisma/adapters-pg avec pilote pg Node.js.

Supabase : deux URL de connexion distinctes — pooler transactionnel PgBouncer (port 6543) pour l'application, connexion directe (port 5432) pour les migrations.

bcrypt (12 tours) : hachage des mots de passe ; class-validator + class-transformer : validation et transformation des données entrantes.

Bitwarden CLI : injection sécurisée des chaînes de connexion ; Docker : reproductibilité de l'environnement ; Git et GitHub : versionnement du schéma et des fichiers SQL de seed.

Documentation officielle : Prisma (relations, migrations, adaptateurs), PostgreSQL (types, index partiels), bcrypt (facteur de coût), Supabase (configuration PgBouncer).

3. Avec qui avez-vous travaillé ?

Conception et implémentation réalisées en autonomie complète. La documentation officielle Prisma, PostgreSQL et Supabase a été centrale, notamment pour la configuration PgBouncer imposée par Supabase (pgbouncer=true, connection_limit=1). Les forums communautaires Prisma et les issues GitHub du projet ont permis de résoudre les problèmes de configuration spécifiques à l'utilisation conjointe de Prisma et Supabase.

4. Contexte

Nom de l'entreprise, organisme ou association **Formation autonome — Projet Vite Gourmand**

Chantier, atelier, service ► Développement full-stack en autonomie

Période d'exercice ► Du 01/01/2026 au 15/02/2026

5. Informations complémentaires (facultatif)

Deux URL Supabase sont utilisées : PgBouncer (port 6543) pour l'applicatif, connexion directe (port 5432) pour les migrations. L'anonymisation RGPD s'effectue en transaction atomique : données personnelles remplacées, sessions effacées, commandes conservées pour les obligations comptables.

Activité-type 2

Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°2 ▶ Développer des composants d'accès aux données SQL et NoSQL

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai conçu et implémenté l'ensemble des composants permettant à l'application de lire, écrire et manipuler les données. L'architecture repose sur deux systèmes de stockage complémentaires : une base de données relationnelle PostgreSQL pour les données transactionnelles et métier, et une base MongoDB pour les données analytiques, les journaux d'audit et les snapshots dénormalisés.

Analyse de l'architecture et choix techniques

Avant de développer les composants, j'ai réfléchi à la répartition des responsabilités entre les deux systèmes. PostgreSQL, grâce à ses contraintes d'intégrité et ses transactions ACID, est adapté à toutes les données nécessitant une cohérence stricte : utilisateurs, commandes, menus, remises, fidélité, avis. MongoDB, grâce à sa flexibilité de schéma et ses performances en écriture, est adapté aux données dont le schéma peut varier et qui n'exigent pas de grosses dépendances de modèles de données : statistiques de tableau de bord, journaux d'activité, instantanés dénormalisés pour les rapports.

Service de connexion Prisma (SQL)

Le premier composant développé est le service de connexion à PostgreSQL. Il étend la classe cliente de Prisma et intègre des crochets de cycle de vie NestJS : connexion établie au démarrage du module, fermeture propre à la destruction du module avec libération du pool de connexions. La connexion utilise le pilote pg de Node.js avec un pool de connexions. En mode développement, toutes les requêtes SQL générées sont enregistrées dans la console. Ce service est déclaré comme fournisseur global dans NestJS, disponible par injection de dépendance dans l'ensemble des modules sans réimportation.

Standardisation de la pagination

Pour garantir la cohérence dans tous les composants de lecture, j'ai défini une fonction utilitaire partagée de pagination. Tous les services retournant des listes exécutent les requêtes de liste et de dénombrement en parallèle avec Promise.all, et formatent le résultat sous la forme d'un objet contenant les éléments et un bloc de métadonnées (page courante, taille, total, nombre de pages).

Client de connexion MongoDB (NoSQL)

Pour MongoDB, j'ai développé un client de connexion implémentant le patron Singleton. Il se connecte au cluster MongoDB Atlas avec l'API serveur en mode strict, un pool de dix connexions simultanées au maximum, une connexion minimum maintenue en permanence et un délai d'inactivité de trente secondes. Des fonctions de connexion, déconnexion et ping sont exposées pour la gestion du cycle de vie.

Sécurisation des composants

L'accès aux données est protégé sur plusieurs niveaux. J'utilise un pipe de validation global inspectant chaque requête (ici, on utilise class-validator, une librairie nestjs), si les règles de validation rejette les propriétés non déclarées il listera chaque champ invalide. Une stratégie JWT Passport extrait et vérifie le token à chaque requête. J'utilise aussi un garde NestJS qui vérifie le rôle sur les routes protégée (voir Fig. 4 et Fig. 6).

Tests et vérification

Pour vérifier le bon fonctionnement des composants, j'ai utilisé une collection Postman couvrant une sélection d'endpoints de l'API, organisée en scénarios end-to-end : inscription, connexion, parcours de commande, gestion des remises, fidélité et vérification des accès non autorisés.

2. Précisez les moyens utilisés :

NestJS : Framework back-end structurant les modules, services, contrôleurs, gardes et intercepteurs.

Prisma ORM : Modélisation du schéma, génération du client TypeScript typé, exécution des requêtes SQL.

@prisma/adapters : Adaptateur natif Prisma pour PostgreSQL avec gestion du pool de connexions.

PostgreSQL / Supabase : Base de données relationnelle cloud (pooler transactionnel PgBouncer pour l'application, pooler de session pour les migrations).

MongoDB Atlas : Base de données NoSQL cloud, driver natif Node.js (sans Mongoose), pool de dix connexions.

class-validator + class-transformer : Validation et transformation des données entrantes via décorateurs TypeScript sur les DTOs.

Passport.js + @nestjs/jwt : Authentification par stratégie JWT, extraction depuis l'en-tête Authorization: Bearer.

bcrypt : Hachage des mots de passe (12 tours de salage).

@nestjs/throttler : Limitation du taux de requêtes (20 par seconde, 100 par dix secondes, 300 par minute).

Swagger / OpenAPI : Documentation interactive générée automatiquement, utilisée pour tester les endpoints.

Postman : Collection de 71 tests automatisés couvrant l'ensemble des cas d'usage de l'API.

Docker : Conteneurisation garantissant la reproductibilité de l'environnement d'exécution.

3. Avec qui avez-vous travaillé ?

Ce projet a été développé de manière entièrement autonome.

4. Contexte

DOSSIER PROFESSIONNEL (DP)

Nom de l'entreprise, organisme ou association

Formation autonome — Projet Vite Gourmand

Chantier, atelier, service

► *Développement full-stack en autonomie*

Période d'exercice

► Du 01/01/2026 au 15/02/2026

5. Informations complémentaires (facultatif)

Activité-type 2

Exemple n°3 ▶

Développer la partie back-end d'une application web ou web mobile sécurisée

Développer des composants métier côté serveur et documenter le déploiement d'une application dynamique web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre du projet Vite Gourmand, j'ai développé l'ensemble des composants constituant la logique métier côté serveur. Le tout avec une architecture modulaire qui est capable d'évoluer sans remettre en cause les composants existants.

Architecture modulaire NestJS

j'ai cartographié les domaines métier et défini un module NestJS pour chacun (voir Fig. 10). Chaque module encapsule son contrôleur, son service et ses DTOs (Data Transfer Object = une structure décrivant la forme des données envoyées ou reçues entre couches API -> contrôleur -> services), et n'expose à l'extérieur que ce qui est nécessaire aux autres modules.

Configuration du point d'entrée

Ici, on utilise le principe de défense en profondeur, souvent appelé **defense in depth** ou le modèle en couche d'oignon. L'idée principale de ce modèle c'est qu'aucune couche ne protège contre tout. Mais si un attaquant perce les premières couches, il tombe sur la deuxième. À chaque couche sa propre difficulté à être percé.

Ainsi avant d'accepter la première requête, le serveur traverse une ligne d'assemblage de sécurité. Chaque étape a un rôle précis.

On force HTTPS, toute requête qui arrive en HTTP est immédiatement redirigée en 308 vers HTTPS. Il n'y a pas de négociation possible.

Concernant la taille des corps de requêtes, ils sont plafonnés à 10 Mo. C'est une protection simple à ajouter contre les requêtes géantes qui viendraient saturer la mémoire du serveur.

Helmet pose les bases des en-têtes de sécurité. C'est une vingtaine d'en-têtes en une ligne, avec interdiction d'embarquer le site dans une iframe grâce à l'attribut (X-Frame-Options: DENY).

On le protège contre le **sniffing** de type MIME, politique de référent stricte, HSTS d'un an en production. La configuration est légèrement ajustée pour autoriser la fenêtre popup de connexion Google OAuth – sans quoi le bouton "Se connecter avec Google" serait bloqué par le navigateur.

Permissions-Policy retire les accès aux capteurs. Le serveur déclare explicitement que l'application n'utilise pas la caméra, le micro ni la géolocalisation. Si un script malveillant tente d'y accéder, le navigateur refuse. On a aussi activé la compression. Les réponses JSON sont compressées en gzip avant d'être envoyées. Cela permet que les pages chargent plus vite, la bande passante est réduite.

Le CORS n'ouvre quant à lui la porte qu'aux origines connues. Seule l'URL du front configurée dans les variables d'environnement peut envoyer des requêtes cross-origin. Les cookies de session sont autorisés dans ces échanges indispensables pour que le JWT HttpOnly voyage automatiquement. On devait aussi se

protéger contre les failles liées aux cookies. C'est ce que permet la protection CSRF. Un cookie `HttpOnly` ne peut être lu par JavaScript, mais un attaquant peut quand même forger une requête qui l'embarque. C'est pourquoi on a utilisé le double-cookie pattern. En effet le front doit lire un second cookie lisible (`vgcsrftoken`) et le renvoyer en header. Sans ça, toute mutation est rejetée avec un 403.

Une protection a aussi été mise pour le préfixe `/api` qui isole les routes backend. Toutes les routes NestJS sont automatiquement préfixées. Par exemple, `/api/orders`, `/api/auth`, etc. Les routes sans ce préfixe renvoient vers le front React en production.

Pour ce qui est de la documentation on la génère au démarrage. À chaque lancement du serveur, NestJS parcourt tous les décorateurs `@ApiTags`, `@ApiOperation`, `@ApiProperty` et construit une interface interactive accessible sur `/api/docs` sans fichiers à maintenir à la main.

Objets de transfert de données (DTOs)

Chaque route de l'API refuse les données dites mal formées avant même d'atteindre le code métier. C'est le rôle des DTOs; des objets qui décrivent exactement ce qu'une route accepte et rejettent tout le reste. Par exemple. Le DTO vérifie six choses dans l'ordre

1. Check l'email
2. Que la politique de mot de passe soit bien respectée
3. Que le prénom soit entre 2 et 200 caractères.
4. Que la case « j'accepte » a été explicitement cochée. Si elle ne l'est pas. L'API refusera la requête avec un message d'erreur clair.

Pour la réinitialisation de mot de passe, les règles sont plus strictes : une expression régulière vérifie que le nouveau mot de passe respecte la politique de mot de passe. Aucun mot de passe faible n'est accepté pour une opération sensible.

Intercepteurs

Avant tout, on se doit de définir un intercepteur, à ne pas confondre avec les exceptions. L'intercepteur exécute avant et après le handler, quand tout se passe bien. `TransformInterceptor` en est le meilleur exemple. Il va recevoir la requête, lance le handler `return next.handle().pipe()`

Après, il reçoit la réponse et l'enveloppe dans une structure JSON uniforme.

L'intercepteur de transformation enveloppe chaque réponse réussie dans une structure JSON uniforme

```
{
  success:false,
  statusCode: number,
  message: string,
  error: string,
  path: string
  timestamp: string,
  details?: unknown
}
```

L'intercepteur de journalisation : il mesure la durée de chaque requête et enregistre méthode les valeurs des attributs de la page de JS. L'intercepteur HTTP alimente le service de logs en mémoire conservant les

cinq cents dernières entrées pour l'interface d'administration.

Filtres d'exceptions

Deux filtres garantissent que toutes les erreurs sont retournées dans le format standard. Le filtre HTTP capture les exceptions NestJS et extrait message, code et détails de validation. Le filtre global capture toutes les exceptions non traitées ; en développement il expose le message réel, en production il renvoie un message générique ne laissant filtrer aucune information sur l'implémentation interne.

Pipes de validation

Quand une requête arrive, deux filtres sont appliqués:

Le premier vérifie la forme des données.

Par exemple, si un champ obligatoire manque, si un email n'est pas valide, si un mot de passe est trop court, tout cela sera rejeté immédiatement. Ce qui est utile : toutes les erreurs sont listées d'un coup. Si l'on envoie un formulaire avec un mdp invalide et email invalide. Tout cela sera rejeté immédiatement. Les champs non déclarés dans le dto sont aussi refusés. Impossible d'injecter des données non prévues. La pipe de parsing, quant à elle, va vérifier que la valeur est un entier dans la plage PostgreSQL MININT $\leq x \leq$ MAX_INT prévenant ainsi les erreurs d'overflow au niveau de la base de données.

Le second filtre concernant les identifiants numériques dans les URLs - /api/orders/42, /api/users/7.

PostgreSQL stocke ces IDs sous forme d'entiers signés 32 bits: si quelqu'un envoie /api/orders/9999999999, la requête dépasse la capacité du type et plante en base. Grâce à ce pipe, on vérifie que la valeur est bien un entier dans la plage autorisée avant d'aller plus loin.

Machine à états des commandes

Le cycle de vie d'une commande suit une machine à états : en attente de confirmation, confirmée, en préparation, prête, en cours de livraison, livrée, ou annulée. Seules certaines transitions sont autorisées. Les modifications (adresse, horaire, instructions spéciales) et l'annulation ne sont possibles que tant que le statut est en attente ou confirmée. Chaque changement de statut est tracé dans une table d'historique avec l'ancien statut, le nouveau statut, les éventuelles notes et l'horodatage. Une nuance est à apporter : via la route admin `POST:id/status`, un employé peut aussi annuler depuis preparing puisque la restriction s'applique uniquement à la route /cancel exposée au client.

Service d'envoi d'emails

Le service email est connecté à un serveur SMTP Titan Email sur le port 465 en TLS implicite, avec vérification de la connexion au démarrage. Chaque envoi produit un fallback en texte brut depuis le HTML pour la compatibilité avec les clients ne gérant pas le HTML. Les envois sont asynchrones et non-bloquants pour les flows auth et contrairement aux envois de newsletters qui sont attendus — c'est intentionnel. L'API répond sans attendre la confirmation d'envoi.

Limitation du taux de requêtes

Un module de limitation de débit est configuré avec trois niveaux : vingt requêtes par seconde contre les pics soudains, cent par dix secondes pour les rafales modérées, trois cents par minute pour la fenêtre longue. Ces limites sont désactivées automatiquement dans l'environnement de test.

Déploiement de la page web dans fly.io

On n'en avait pas parlé jusque-là, fly.io a été choisi à la place d'autres comme vercel simplement parce qu'il a été le premier à me tomber dans les mains. De toute façon il fallait que j'apprenne l'un ou l'autre. Il n'y a pas de choix purement technique derrière sa sélection. Fly.io fait partie de ce que l'on appelle un PAAS qui déploie des containers Docker directement sur des machines bare-metal. L'avantage c'est qu'on n'a pas de serveur à gérer, HTTPS automatique grâce au protocole Let's Encrypt.

Le build se fait en 3 étapes dans un seul Dockerfile:

1. *npm run build pour le front*
2. *Backend prisma generate + npm run build*
3. *En runtime, l'image finale est allégée en utilisant ~alpine*

L'image finale contient uniquement les dépendances de production. Le frontend compilé est copié dans `/app/public` et servi directement par NestJS – un seul container, un seul port (8080).

Pour connecter les requêtes d'API du front, à la place d'utiliser une API_URL, on utilise volontairement des requêtes relatives /api/... — le frontend n'a donc pas besoin de connaître le domaine.

La configuration était finalement fastidieuse.

On a configuré force_https=true avec Fly, qui redirige automatiquement HTTP vers HTTPS. Avantage : pas de CA.

Toutes les 15 secondes, on vérifie l'état du serveur web via des requêtes comme GET /api.

VM : 1 CPU partagé, 1 Go de RAM.

Et on a stocké les secrets d'environnements dans fly.io avec la commande flyctl secrets set.

L'image tourne avec un utilisateur non-root nestjs (uid 1001) et tini est utilisé en PID 1 pour gérer correctement les signaux UNIX (SIGTERM → arrêt propre du process Node).

Enfin, flyctl est encapsulé dans un service Docker Compose pour ne pas avoir à l'installer sur la machine hôte. (voir Fig.21).

Tests et documentation

L'API est documentée via Swagger et testée avec une collection Postman de soixante et onze tests couvrant les parcours nominaux et les cas d'erreur, dont la vérification que les routes protégées rejettent les requêtes non authentifiées ou insuffisamment autorisées. J'utilise JSDoc pour commenter les fonctions.

2. Précisez les moyens utilisés :

DOSSIER PROFESSIONNEL (DP)

NestJS : Framework back-end structurant modules, contrôleurs, services, gardes, intercepteurs, filtres et pipes.

TypeScript : Typage strict de l'ensemble des composants (DTOs, payloads JWT, requêtes Express étendues).

class-validator + class-transformer : Décorateurs de validation et transformation sur les DTOs.

Passport.js + @nestjs/passport : Stratégies d'authentification JWT et Google OAuth 2.0.

@nestjs/jwt : Génération et vérification des tokens JWT (accès 15 minutes, réinitialisation 1 heure).

bcrypt : Hachage des mots de passe avec 12 tours de salage, vérification sécurisée.

@nestjs/throttler : Limitation du taux de requêtes à trois niveaux (court, moyen, long).

@nestjs/cache-manager : Cache en mémoire global, TTL 60 secondes, maximum 100 entrées.

Nodemailer : Envoi d'emails via SMTP Titan Email (port 465, TLS implicite).

Helmet : En-têtes HTTP de sécurité, configuration adaptée pour Google OAuth popup.

Compression : Compression des réponses HTTP pour réduire la bande passante.

Swagger / OpenAPI : Documentation interactive générée depuis les décorateurs NestJS.

Postman : Collection de 71 tests automatisés couvrant l'ensemble des endpoints.

Docker : Conteneurisation garantissant la reproductibilité de l'environnement.

3. Avec qui avez-vous travaillé ?

L'ensemble des composants métier côté serveur a été développé de manière autonome. Cette posture m'a conduit à prendre en charge simultanément la définition de l'architecture (choix des patterns NestJS, organisation des modules, stratégie de sécurité), le développement des composants, et la validation de leur comportement par les tests. Sans équipe pour challenger les décisions, j'ai fait appel à la documentation officielle NestJS et aux guides de bonnes pratiques de la communauté pour valider mes choix architecturaux — notamment sur la configuration des gardes globaux, la gestion des erreurs standardisée et la mise en place de l'authentification multi-stratégie.

4. Contexte

Nom de l'entreprise, organisme ou association **Formation autonome — Projet Vite Gourmand**

Chantier, atelier, service ▶ Développement full-stack en autonomie

Période d'exercice ▶ Du 01/01/2026 au 15/02/2026

5. Informations complémentaires (facultatif)

DOSSIER PROFESSIONNEL (DP)

Chaque requête traverse une chaîne ordonnée : rate limiting → JWT → rôle → validation DTO → service métier. Cette architecture est extensible sans modifier les contrôleurs existants. La conformité RGPD est imposée au niveau du DTO : le consentement doit valoir exactement true, sans possibilité de contournement.

DOSSIER PROFESSIONNEL (DP)

Titres, diplômes, CQP, attestations de formation

Intitulé	Autorité ou organisme	Date
Baccalauréat scientifique	Académie de Reims – Éducation nationale	01/07/2017
Attestation de résultats académiques – Coursus Common Core (niveau 4.62 / 21), 42 Madrid	42 Madrid Fundación Telefónica	25/05/2026
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.

DOSSIER PROFESSIONNEL (DP)

Déclaration sur l'honneur

Je soussigné*LESIEUR.DYLAN*.....,
déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je
suis l'auteur des réalisations jointes.

Fait à *Madrid*..... le *21/05/2026*.....
pour faire valoir ce que de droit.

Signature :

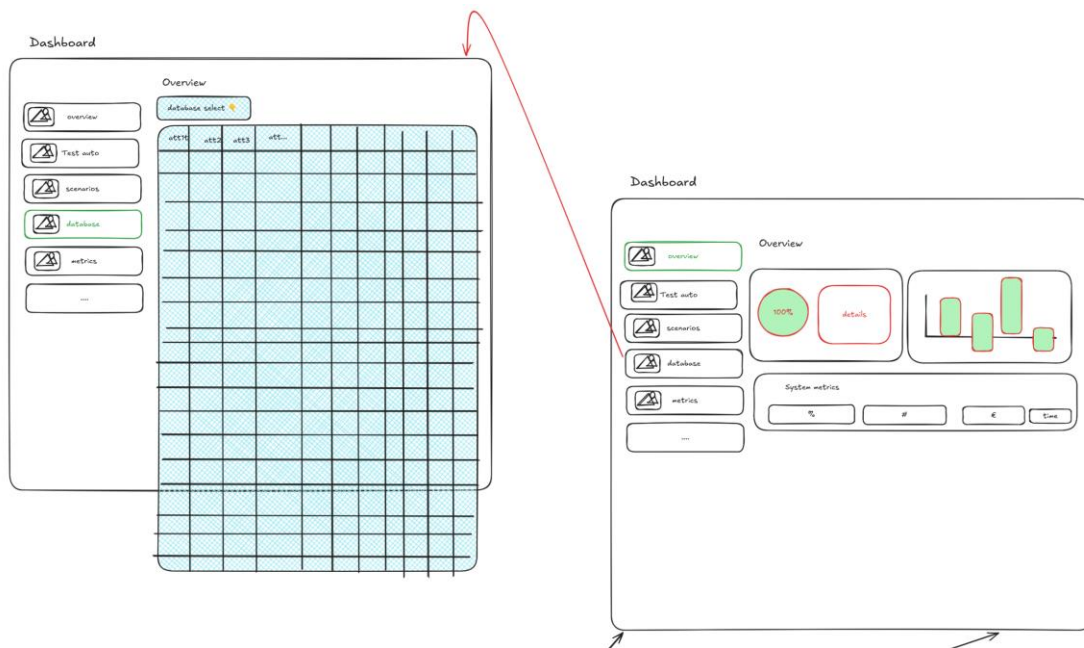


Documents illustrant la pratique professionnelle

(facultatif)

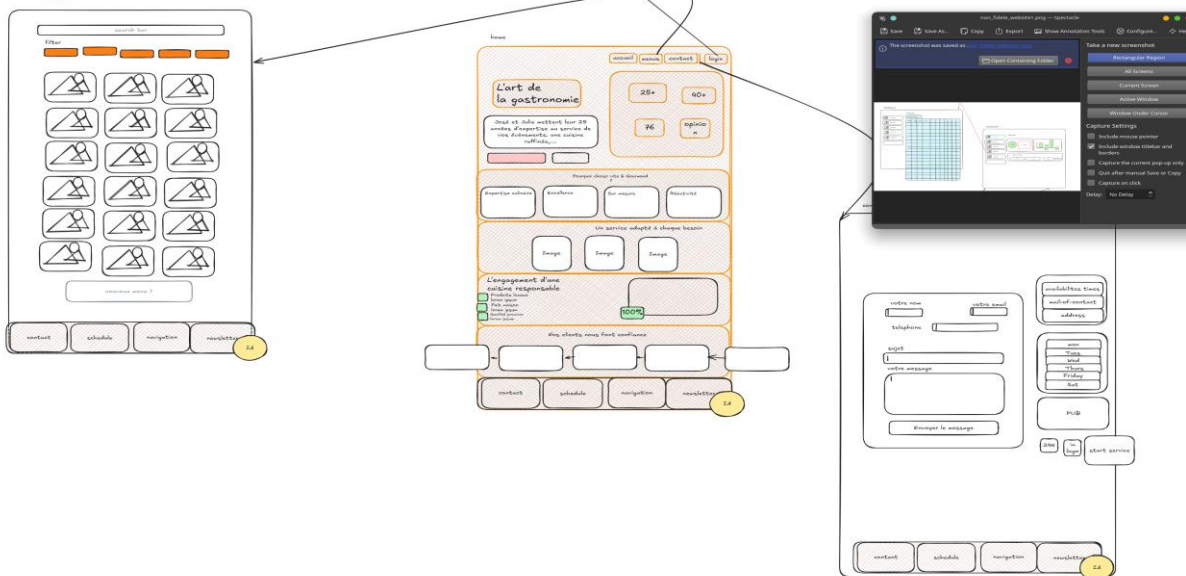
Intitulé
Fig. 1 — Maquette basse-fidélité -- Interface administrateur de DevBoard, un espace admin, employé, client
Fig. 2 — Maquette basse-fidélité -- interface côté client avec 3 pages, home, menus, contact
Fig. 3 — relation de base de donnée fidélité et marketing
Fig. 4 — Schéma conceptuel de la base de données relationnelle pour l'identité,role,sécurité
Fig. 5 — Diagramme d'entité relations des menus à l'ingrédient
Fig. 6 — Matrice acteur × cas d'utilisation (droits par rôle)
Fig. 7 — Diagramme de séquence d'une prise de commande.
Fig. 8 — container running au travers du makefile target qui orchestre docker-compose pour libérer rapidement la mémoire et pour activer le programme dans son propre environnement isoler.
Fig. 9 — détail de l'orchestration des containers en train de se développer
Fig. 10 — Cycle de vie d'une requête NestJS : architecture en couches successives illustrant le traitement top-down de la requête HTTP entrante jusqu'à la réponse retournée au client
Fig. 11 — QA tester sonarcloud overview du contexte du projet à la date du 26 mai 2025
Fig. 12 — lighthouse test du site web réel de la page home
Fig. 13 — lighthouse test du site web réel de la page menu
Fig. 14 — CI Pipeline de github Actions avec des test vérifiés et fonctionnels
Fig. 15 — mindmap cartographié de l'architecture de l'application et des spécificatoinis sous-jacents
Fig. 16 — Extraits des tokens css que l'on utilise côté front-end
Fig. 17 — nouvelle couche archtiecturelle docker des micro-services
Fig. 18 — extraits du robot text qui permet de dialoguer directement avec les robots des navigateurs
Fig. 19 — extrait de code qui montre l'algorithme et l'usage de bcrypt qui est salé 12 fois
Fig. 20 — preuve d'aggrégation de la propriété pgbouncer pour connecter avec supabase parce qu'ils utilisent une connexion stricte simultannées. Sans pooler le server saturerait rapidement.
Fig. 21 — extrait de mon code d'orchestration makefile qui manage fly.io

DOSSIER PROFESSIONNEL (DP)

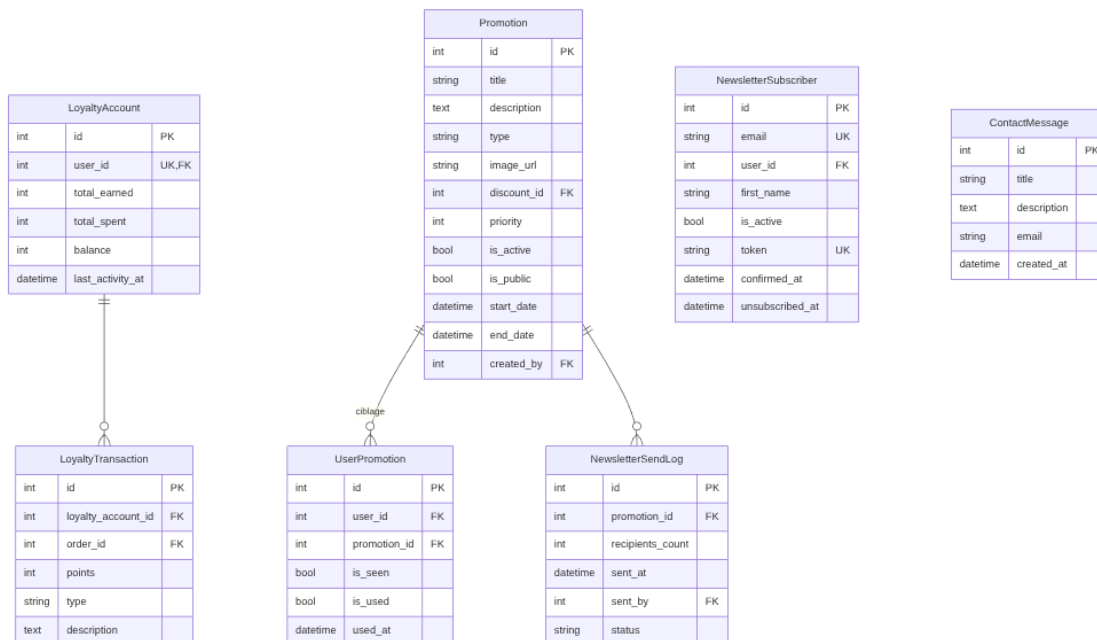


ANNEXES

(Si le RC le prévoit)

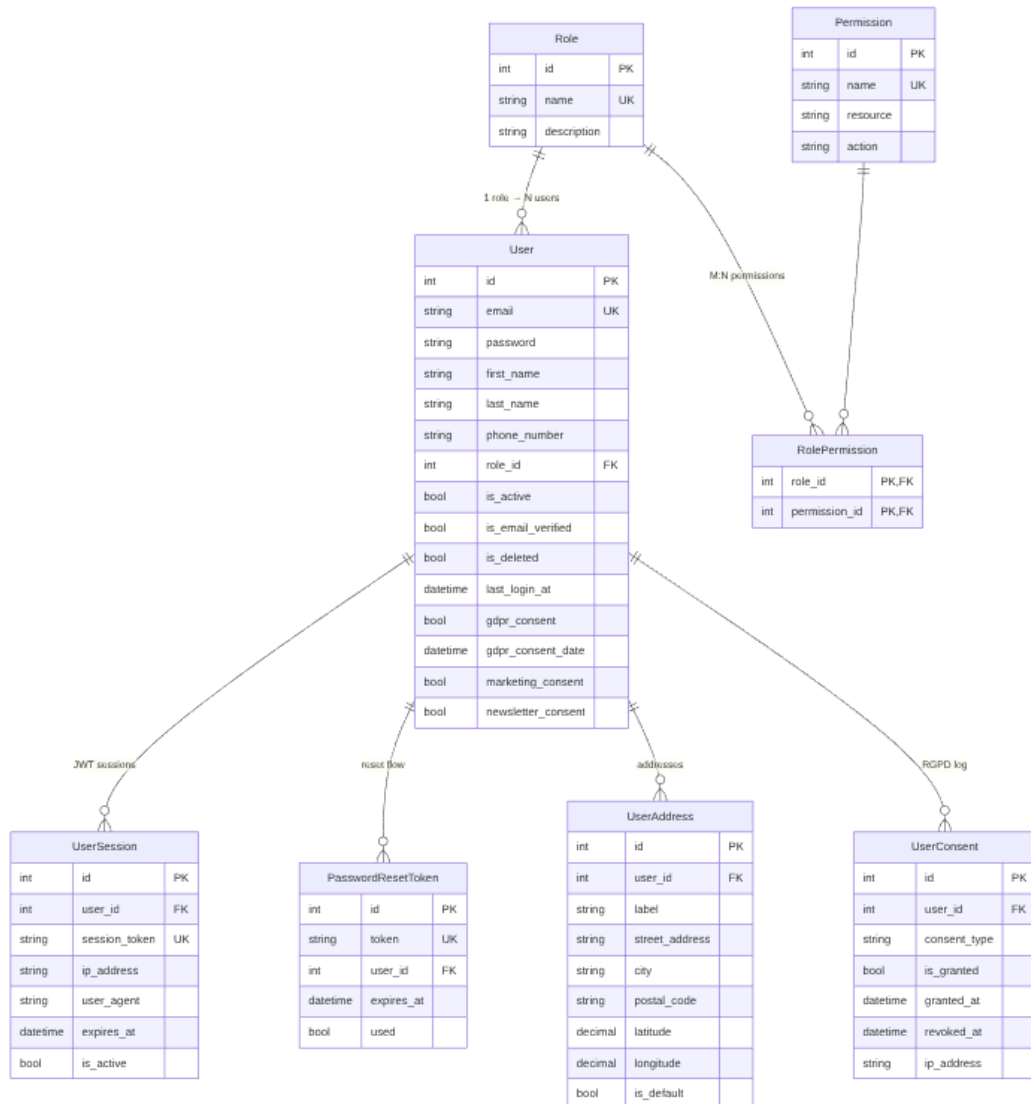


2.4 Fidélité & marketing

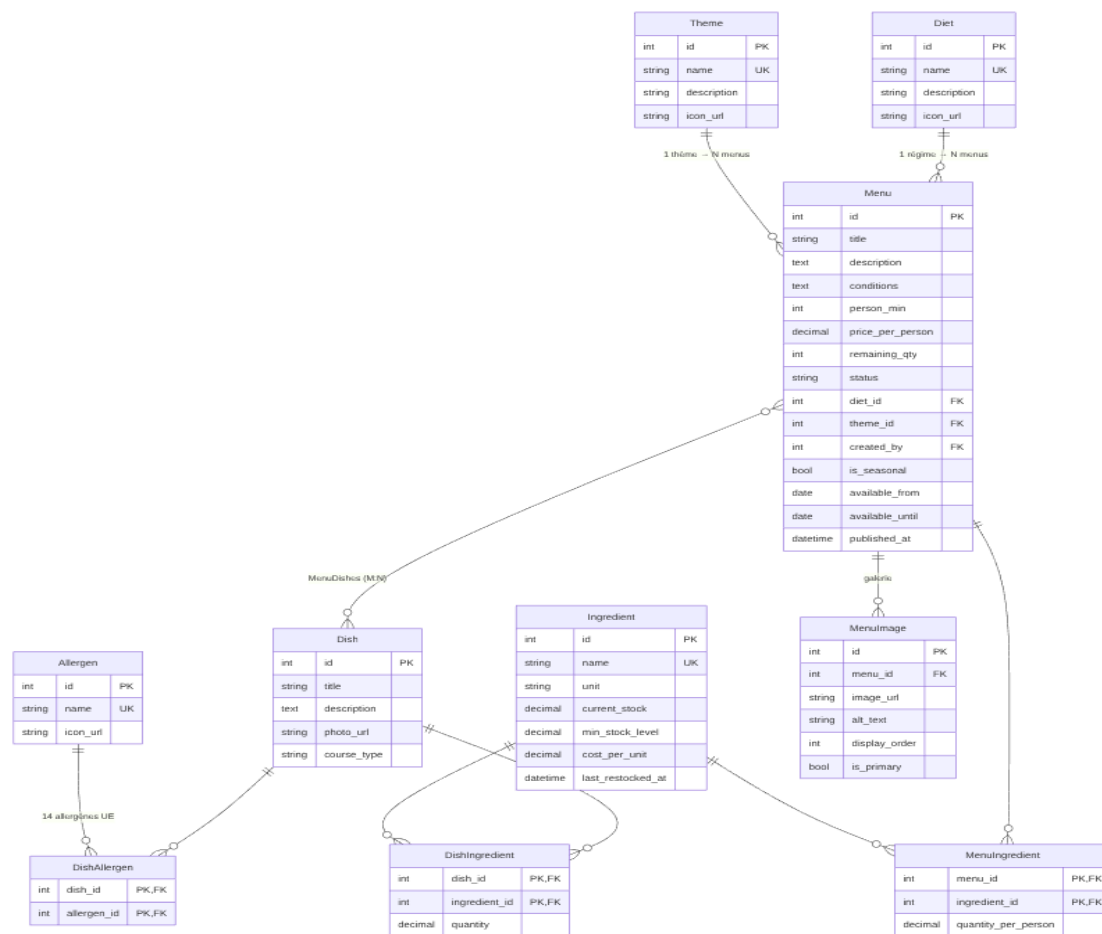


2. Vue par domaine

2.1 Identité, sessions, sécurité

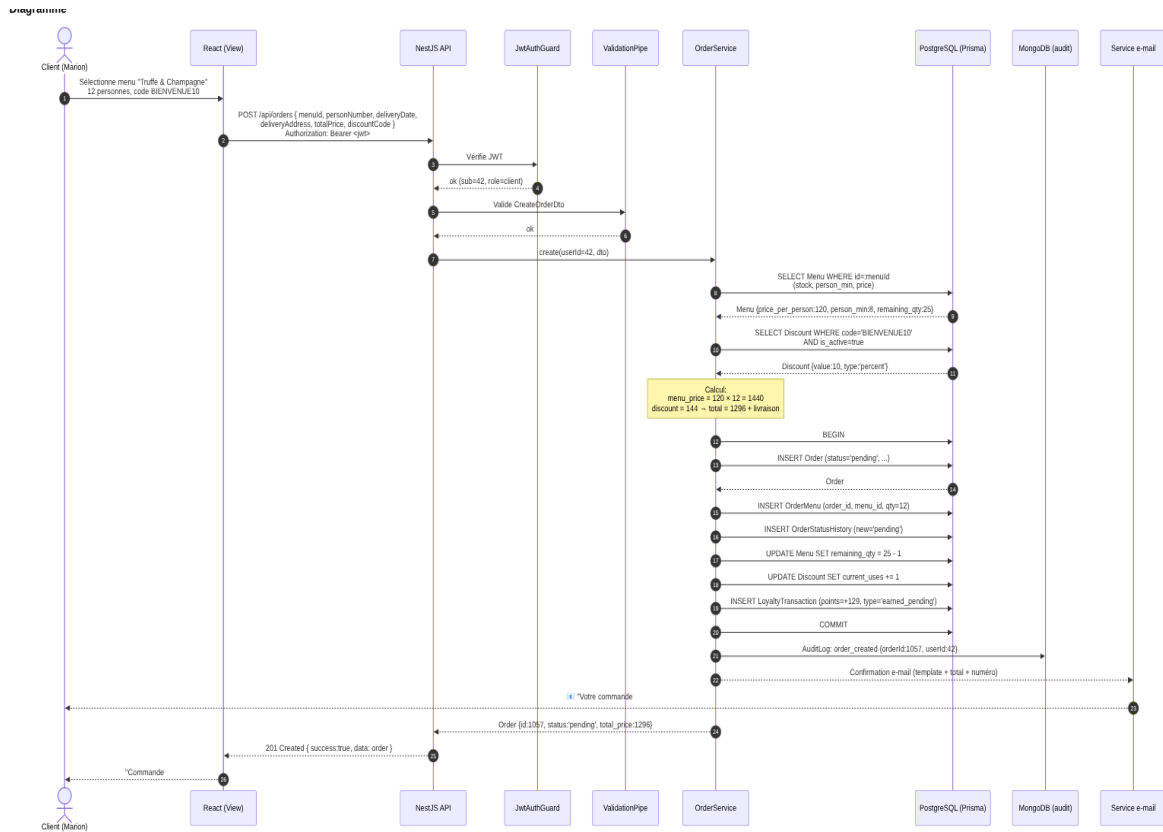


2.2 Catalogue : menus, plats, allergènes, ingrédients



4. Matrice acteur × cas d'utilisation (extrait)

Cas d'utilisation	Visiteur	Client	Employé	Admin
Consulter menus	✓	✓	✓	✓
Envoyer formulaire contact	✓	✓	—	—
Créer un compte	✓	—	—	—
Passer une commande	—	✓	—	—
Annuler avant confirmation	—	✓	—	—
Modifier statut commande	—	—	✓	✓
Annuler commande confirmée	—	—	✓	✓
Modérer un avis	—	—	—	✓
Créer un menu	—	—	—	✓
Créer un employé	—	—	—	✓
Lancer une newsletter	—	—	—	✓
Consulter analytics	—	—	—	✓
Demander suppression RGPD	—	✓	—	—
Approuver suppression RGPD	—	—	—	✓



Points techniques

dlesieur@dlesieur:~/Documents/vite-gourmand\$ make turn-off

SHUTTING DOWN VITE GOURMAND

- Stopping frontend...
✓ Frontend stopped
- Stopping backend...
✓ Backend stopped
- ✓ Everything shut down.

● **dlesieur@dlesieur:~/Documents/vite-gourmand\$ make turn-on**

TURNING ON VITE GOURMAND

- 🔧 Starting Backend (NestJS) on port 3000...
PID: 8929
- 💻 Starting Frontend (Vite) on port 5173...
PID: 8936
- ✓ Servers started!
- 💻 Frontend → http://localhost:5173
- 🔧 Backend → http://localhost:3000/api

dlesieur@dlesieur:~/Documents/vite-gourmand\$

```
d@esieur@d@esieur:~/Documents/vite-gourmand$ make all

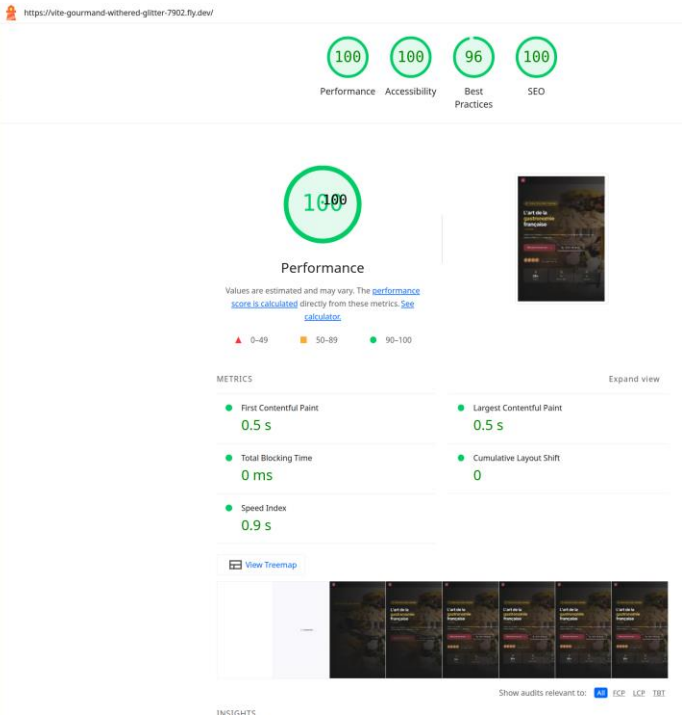
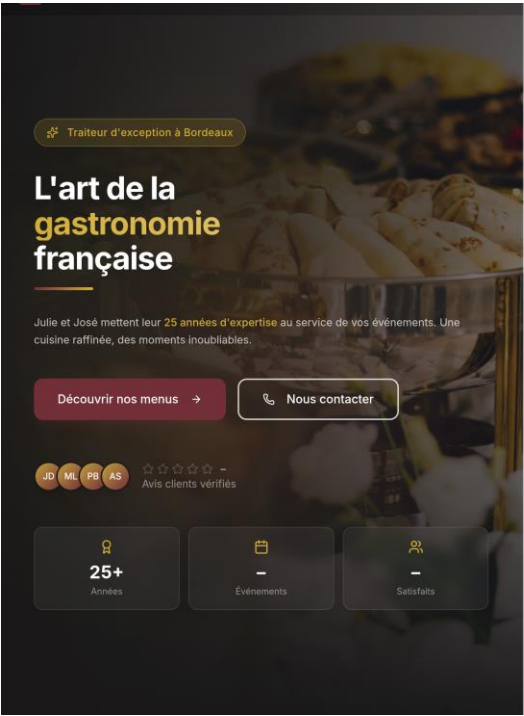
VITE GOURMAND - Containerized Development

Everything runs inside Docker containers.
Your host machine only needs Docker installed!

Steps:
1. 🚀 Build dev container (Node.js 22 Alpine)
2. 📁 Fetch .env from Bitwarden (interactive)
3. 📦 Install dependencies inside container
4. 🏗️ Generate Prisma client & compile TypeScript
5. 🏃 Start dev servers inside container

Building Development Container

Service dev Building
Sending build context to Docker daemon 60.09MB
Step 1/11 : FROM node:22-alpine
--> 968df39aedece
Step 2/11 : RUN apk add --no-cache bash curl git jq make psmisc procps python3 g++ openssl libc6-compat
--> Running in b672c5adceb6
( 1/53) Installing ncurses-terminfo-base (6.5_p20251123-r0)
( 2/53) Installing libncursesw (6.5_p20251123-r0)
( 3/53) Installing readline (8.3.1-r0)
( 4/53) Installing bash (5.3.3-r1)
Executing bash-5.3.3-r1.post-install
( 5/53) Installing brotli-libs (1.2.0-r0)
( 6/53) Installing c-ares (1.34.6-r0)
( 7/53) Installing libunistring (1.4.1-r0)
( 8/53) Installing libidn2 (2.3.8-r0)
( 9/53) Installing nghttp2-libs (1.69.0-r0)
(10/53) Installing libpsl (0.21.5-r3)
(11/53) Installing zstd-libs (1.5.7-r2)
(12/53) Installing libcurl (8.19.0-r0)
(13/53) Installing curl (8.19.0-r0)
(14/53) Installing libstdc++-dev (15.2.0-r2)
(15/53) Installing jansson (2.14.1-r0)
(16/53) Installing binutils (2.45.1-r0)
(17/53) Installing libgomp (15.2.0-r2)
(18/53) Installing libatomic (15.2.0-r2)
(19/53) Installing gmp (6.3.0-r4)
(20/53) Installing isl26 (0.26-r1)
(21/53) Installing mpfr4 (4.2.2-r0)
(22/53) Installing mpcl (1.3.1-r1)
(23/53) Installing gcc (15.2.0-r2)
(24/53) Installing musl-dev (1.2.5-r23)
(25/53) Installing g++ (15.2.0-r2)
(26/53) Installing libexpat (2.7.5-r0)
(27/53) Installing pcre2 (10.47-r0)
(28/53) Installing git (2.52.0-r0)
(29/53) Installing git-init-template (2.52.0-r0)
(30/53) Installing oniguruma (6.9.10-r0)
(31/53) Installing jq (1.8.1-r0)
(32/53) Installing musl-obstack (1.2.3-r2)
(33/53) Installing libucontext (1.3.3-r0)
(34/53) Installing gcompat (1.1.0-r4)
(35/53) Installing make (4.4.1-r3)
(36/53) Installing openssl (3.5.6-r0)
(37/53) Installing libintl (0.24.1-r1)
(38/53) Installing skelibs-libs (2.14.4.0-r0)
(39/53) Installing utmps-libs (0.1.3.1-r0)
(40/53) Installing libproc2 (4.0.5-r0)
(41/53) Installing procps-ng (4.0.5-r0)
(42/53) Installing psmisc (23.7-r0)
(43/53) Installing libbz2 (1.0.8-r6)
(44/53) Installing libffi (3.5.2-r0)
```

🏠

Nos Menus

Des créations culinaires d'exception, pensées pour sublimer chacun de vos événements. Filtrez par thème, régime alimentaire ou budget.

● ● ● ● ●

🔍 Rechercher un menu, un thème...

Filtres

Tous

Anniversaire

Baptême

Barbecue

Brunch

Cocktail

Entreprise

6 menus disponibles

📷 Mariage

Menu Végétarien Élégant

végétarien

3 allergies

Alternative végétarienne raffinée

Min. 20 pers. 65 €/pers.

📷 Entreprise

Cocktail Entreprise

classique

5 allergies

Assortiment de canapés et mignardises

Min. 30 pers. 45 €/pers.

Voir le détail →

Voir le détail →

Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

← Pipeline

update #21

Summary

All jobs

1 - Lint (ESLint)

2 - Unit tests (Jest)

3 - E2E tests (Jest + DB)

4 - Custom validation tests

5 - Postman API tests

7 - Frontend security

6 - Test summary (all suites)

8 - Build (NestJS + Vite)

9 - SonarCloud

Run details

Usage

Workflow file

Triggered via push 6 minutes ago

Status

Total duration

Artifacts

LEslyan pushed → 836f6c1

Success

6m 30s

8

ci-cd.yml

on: push

1 - Lint (ESLint) 44s

2 - Unit tests (Jest) 37s

3 - E2E tests (Jest + DB) 2m 36s

4 - Custom validation tests 24s

5 - Postman API tests 1m 38s

7 - Frontend security 14s

6 - Test summary (all suites) 28s

8 - Build (NestJS + Vite) 53s

9 - SonarCloud 2m 7s

6 - Test summary (all suites) summary

Test Summary

Suite	Passed	Failed	Total
Jest unit	324	0	324
Jest e2e	346	0	346
Custom validation	153	0	153
Postman API	71	0	71
Total	894	0	894

https://vite.gourmand.withered-glitter-7902.fly.dev/menus

91

Performance

98

Accessibility

92

Best Practices

100

SEO

91

Performance

Values are estimated and may vary. The [performance score is calculated](#) directly from these metrics. [See calculation.](#)

▲ 0-49

■ 50-89

● 90-100

METRICS

Expand view

First Contentful Paint 2.2 s

Largest Contentful Paint 2.3 s

Total Blocking Time 0 ms

Cumulative Layout Shift 0

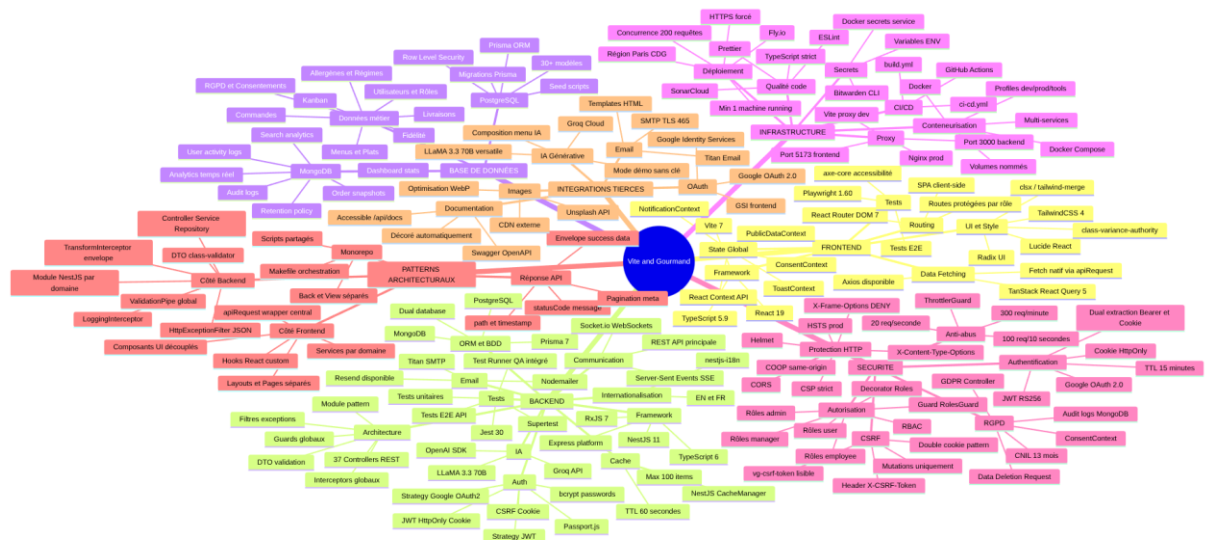
Speed Index 6.0 s

View Treemap

Show audits relevant to: LCP TBT

DOSSIER PROFESSIONNEL-Version du 01/06/2016

Page 40



```

View > src > styles > # graphical_chart_part2.css > .root
You, yesterday | 1 author (You)

1  /**
2  * DevBoard Design System - Typography & Spacing
3  * Continuation of foundation variables
4  */
5
6  :root {
7    /*
8     * TYPOGRAPHY
9     */
10
11    /* Font Families */
12    --font-family-sans: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;
13    --font-family-mono: 'JetBrains Mono', 'Fira Code', 'Consolas', monospace;
14
15    /* Font Sizes */
16    --font-size-2xs: 0.5rem; /* 8px */
17    --font-size-xs2: 0.625rem; /* 10px */
18    --font-size-xs1: 0.65rem; /* ~10.4px */
19    --font-size-xs0: 0.7rem; /* ~11.2px */
20    --font-size-xs: 0.75rem; /* 12px */
21    --font-size-xs-plus: 0.8rem; /* ~12.8px */
22    --font-size-sm-minus: 0.82rem; /* ~13.1px */
23    --font-size-sm-alt: 0.8125rem; /* 13px */
24    --font-size-sm-near: 0.85rem; /* ~13.6px */
25    --font-size-sm: 0.875rem; /* 14px */
26    --font-size-sm-plus: 0.9rem; /* ~14.4px */
27    --font-size-base-sm: 0.95rem; /* ~15.2px */
28    --font-size-base: 1rem; /* 16px */
29    --font-size-base-lg: 1.1rem; /* ~17.6px */
30    --font-size-lg: 1.125rem; /* 18px */
31    --font-size-xl: 1.25rem; /* 20px */
32    --font-size-2xl: 1.5rem; /* 24px */
33    --font-size-2xl-lg: 1.75rem; /* 28px */
34    --font-size-3xl: 1.875rem; /* 30px */
35    --font-size-4xl: 2.25rem; /* 36px */
36    --font-size-5xl: 2.5rem; /* 40px */
37    --font-size-6xl: 3rem; /* 48px */
38
39    /* Font Weights */
40    --font-weight-light: 300;
41    --font-weight-normal: 400;
42    --font-weight-medium: 500;
43    --font-weight-semibold: 600;
44    --font-weight-bold: 700;
45    --font-weight-extrabold: 800;
46
47    /* Line Heights */
48    --line-height-tight: 1.25;
49    --line-height-snug: 1.35;
50    --line-height-normal: 1.5;
51    --line-height-relaxed: 1.625;
52    --line-height-loose: 2;
53    --line-height-1-1: 1.1;
54    --line-height-1-15: 1.15;
55    --line-height-1-2: 1.2;
56    --line-height-1-3: 1.3;
57    --line-height-1-4: 1.4;
58    --line-height-1-7: 1.7;
59    --line-height-1-75: 1.75;
60
61    /* Letter Spacing */
62    --letter-spacing-tight: -0.025em;
63    --letter-spacing-normal: 0;
64    --letter-spacing-wide: 0.025em;
65
66    /*
67     * SPACING SCALE
68     */
69  }

```

```

dlesieur — .../vite-gourmand/infrastructure — main --- 01:07:20
> tree -d
.
├── contracts
├── services
│   ├── app
│   │   ├── config
│   │   └── scripts
│   ├── dev
│   │   ├── config
│   │   └── scripts
│   ├── fly
│   │   ├── config
│   │   └── scripts
│   ├── mongo
│   │   ├── config
│   │   └── scripts
│   ├── postgres
│   │   ├── config
│   │   └── scripts
│   ├── secrets
│   │   ├── config
│   │   └── scripts
└── sonarcloud
    └── results

23 directories

```

```

User-agent: *
Allow: /
Disallow: /admin
Disallow: /dashboard
Disallow: /employee

Sitemap: https://vite-gourmand.fr/sitemap.xml

```

```

import { Injectable, UnauthorizedException } from '@nestjs/common';
import * as bcrypt from 'bcrypt';

const SALT_ROUNDS = 12;

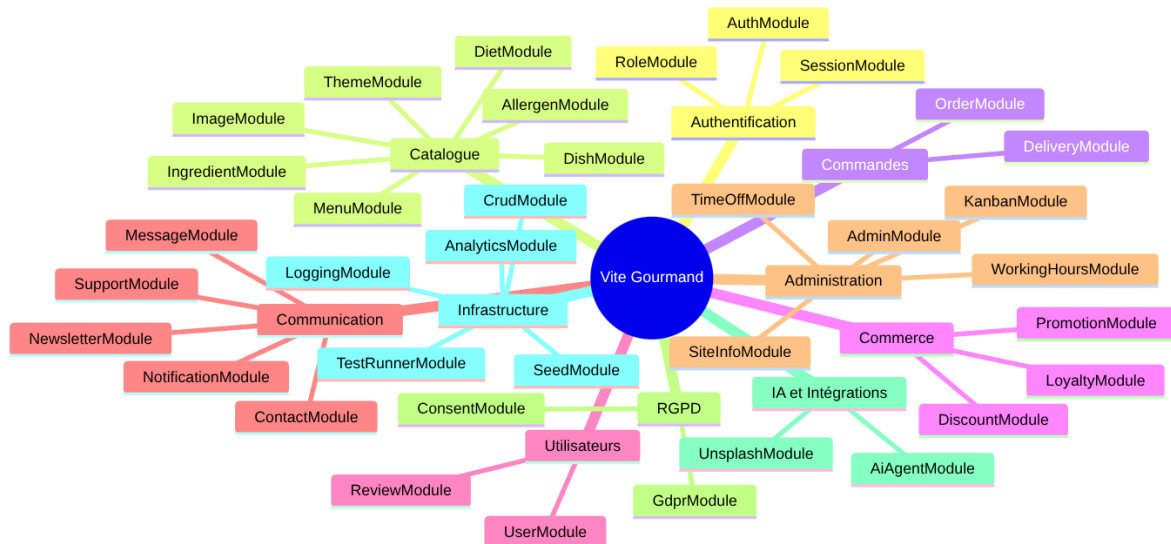
You, yesterday | 1 author (You)
@Injectable()
export class PasswordService {
  /**
   * Hash a plain text password
   */
  async hash(password: string): Promise<string> {
    return bcrypt.hash(password, SALT_ROUNDS);
  }

  /**
   * Verify password against hash
   * @throws UnauthorizedException if password doesn't match
   */
  async verify(password: string, hash: string): Promise<void> {
    const isValid = await bcrypt.compare(password, hash);
    if (!isValid) {
      throw new UnauthorizedException('Invalid credentials');
    }
  }
}

```

You, 3 months ago • new modules created based on legacy code with n...

1. Vue d'ensemble — 10 domaines



```

90     ThrottlerModule.forRoot({
91       throttlers:
92         process.env.NODE_ENV === 'test'
93           ? []
94           : [
95             {
96               name: 'short',
97               ttl: 1000, // 1 second
98               limit: 20, // 20 requests per second (SPA fires parallel calls on load)
99             },
100             {
101               name: 'medium',
102               ttl: 10000, // 10 seconds
103               limit: 100, // 100 requests per 10 seconds
104             },
105             {
106               name: 'long',
107               ttl: 60000, // 1 minute
108               limit: 300, // 300 requests per minute
109             },
110           ],
111     }),

```

```

# Construct connection URLs
# Using Transaction pooler (port 6543) for application connections
DATABASE_URL="postgresql://postgres.${SUPABASE_PROJECT_REF}:${ENCODED_PASSWORD}
@aws-0-eu-west-3.pooler.supabase.com:6543/postgres?pgbouncer=true"

```

```

1  ##@ Deployment
2
3  .PHONY: deploy-fly deploy-check deploy-status deploy-logs deploy-certs deploy-fly-shell deploy-safe
4
5  deploy-fly: ## Deploy backend to Fly.io
6      @$(SCRIPTS_PATH)/deploy/fly.sh
7
8  deploy-check: ## Run pre-deployment checks (compile, tests, env)
9      @$(SCRIPTS_PATH)/deploy/check.sh
10
11  deploy-status: ## Show current Fly.io application status
12      @$(SCRIPTS_PATH)/deploy/status.sh
13
14  deploy-logs: ## Stream live Fly.io application logs
15      @$(SCRIPTS_PATH)/deploy/logs.sh
16
17  deploy-certs: ## Inspect/request Fly managed certificates (CREATE_CERTS=true to create)
18      @CREATE_CERTS="$(CREATE_CERTS)" HOSTS="$(HOSTS)" $(SCRIPTS_PATH)/deploy/fly-certificates.sh
19
20  deploy-fly-shell: ## Open a shell in the Dockerized Fly CLI service
21      @if [ -f .env.production ]; then \
22          $(DOCKER_COMPOSE) --env-file .env.production --profile tools run --rm fly; \
23      else \
24          $(DOCKER_COMPOSE) --profile tools run --rm fly; \
25      fi
26
27  deploy-safe: deploy-check deploy-fly ## Run pre-checks then deploy to Fly.io
28

```